

Binäre Suche – Integer-Array (Winter 2021/22)

Pseudocode (IHK-Stil):

```
Funktion binarySearch(werte: Array<Integer>, x: Integer) → Integer
    links = 0
    rechts = länge(werte) - 1
    while links ≤ rechts
        mitte = (links + rechts) / 2
        if werte[mitte] == x dann
            return mitte
        else if werte[mitte] < x dann
            links = mitte + 1
        else
            rechts = mitte - 1
        end if
    end while
    return -1
end Funktion
```

Erklärung:

Voraussetzung: werte ist aufsteigend sortiert, sonst liefert die Binärsuche falsche Ergebnisse.

Idee: In jedem Schritt halbiert du das Suchintervall. Vergleiche x mit dem mittleren Element:

- Gleich → gefunden, Index zurückgeben.
- Kleiner → rechts-Grenze nach links verschieben ($\text{rechts} = \text{mitte} - 1$).
- Größer → links-Grenze nach rechts verschieben ($\text{links} = \text{mitte} + 1$).

Terminierung: Sobald $\text{links} > \text{rechts}$, existiert x nicht im Array → -1.

Komplexität: $O(\log n)$ Zeit, $O(1)$ Speicher.

Typische Fehler: Vergessen, dass Daten sortiert sein müssen; Off-by-one bei Grenzen; ganzzahlige Mitte.

Binäre Suche – Kurse nach Kursnummer (Winter 2020/21)

Pseudocode (IHK-Stil):

```
Funktion sucheKurs(kurse: Array<Kurs>, nr: Integer) → Kurs
    links = 0
    rechts = länge(kurse) - 1
    while links ≤ rechts
        mitte = (links + rechts) / 2
        if kurse[mitte].getKursnummer() == nr dann
            return kurse[mitte]
        else if kurse[mitte].getKursnummer() < nr dann
            links = mitte + 1
        else
            rechts = mitte - 1
        end if
    end while
    return null
end Funktion
```

Erklärung:

Objektvariante: Statt primitiver Werte vergleichst du über einen Schlüssel (here: `getKursnummer()`).

Voraussetzung: Das Array `kurse` ist nach Kursnummer sortiert (aufsteigend).

Rückgabe: Das Kurs-Objekt oder null, wenn nicht gefunden.

Prüfungstipp: Den Schlüssel immer explizit nennen (z. B. „Vergleich über `getKursnummer()`“).

Binäre Suche – Bestellungen nach Nummer (Winter 2022/23)

Pseudocode (IHK-Stil):

```
Funktion findeBestellung(bestellungen: Array<Bestellung>, nr: Integer) → Bestellung
    l = 0
    r = länge(bestellungen) - 1
    while l ≤ r
        m = (l + r) / 2
        if bestellungen[m].getBestellnummer() == nr dann
            return bestellungen[m]
        else if bestellungen[m].getBestellnummer() < nr dann
            l = m + 1
        else
            r = m - 1
        end if
    end while
    return null
end Funktion
```

Erklärung:

Gleiche Idee wie bei Kursen, nur mit Bestellungen. Wichtig ist die Sortierung nach Bestellnummer.

Edge Cases: Leeres Array; gesuchte Nummer kleiner als erstes Element oder größer als letztes.

Fehlerquelle: Vergleich auf falsches Feld oder unsortierte Eingabe.

Binäre Suche – generisch per Schlüssel-Funktion (IHK-Stil, häufige Abwandlung)

Pseudocode (IHK-Stil):

```
Funktion binarySearchByKey(L: Liste<T>, key: Funktion<T → Integer>, such: Integer)
    l = 0
    r = L.size() - 1
    while l ≤ r
        m = (l + r) / 2
        k = key(L.get(m))
        if k == such dann
            return m
        else if k < such dann
            l = m + 1
        else
            r = m - 1
        end if
    end while
    return -1
end Funktion
```

Erklärung:

Generisch: Du übergibst eine key-Funktion, die den Sortierschlüssel liefert (z. B. Kurs → Kursnummer).

Vorteil: Eine Binärsuch-Implementierung für viele Typen verwendbar.

Prüfungstipp: In der Aufgabenstellung steht oft „nutzen Sie Function“ oder „Schlüssel-funktion“.

Binäre Suche – erstes Vorkommen bei Duplikaten (Variante)

Pseudocode (IHK-Stil):

```
Funktion binarySearchErstes(werte: Array<Integer>, x: Integer) → Integer
    links = 0
    rechts = länge(werte) - 1
    ergebnis = -1
    while links ≤ rechts
        mitte = (links + rechts) / 2
        if werte[mitte] == x dann
            ergebnis = mitte
            rechts = mitte - 1      // weiter links suchen
        else if werte[mitte] < x dann
            links = mitte + 1
        else
            rechts = mitte - 1
        end if
    end while
    return ergebnis
end Funktion
```

Erklärung:

Ziel: Bei mehrfach vorkommendem x den linken (ersten) Index finden.

Trick: Bei Gleichheit nicht sofort beenden, sondern Ergebnis merken und weiter links suchen.

Analog gibt es binarySearchLetztes für das letzte Vorkommen (rechts weitersuchen).

Binäre Suche – Einfügeposition (lowerBound)

Pseudocode (IHK-Stil):

```
Funktion lowerBound(werte: Array<Integer>, x: Integer) → Integer
    links = 0
    rechts = länge(werte)
    while links < rechts
        mitte = (links + rechts) / 2
        if werte[mitte] < x dann
            links = mitte + 1
        else
            rechts = mitte
        end if
    end while
    return links
end Funktion
```

Erklärung:

Lower Bound gibt die erste Position zurück, an der x eingefügt werden kann, ohne die Sortierung zu brechen.

Nützlich für „nicht gefunden → wo einfügen?“. Auch Basis für Binsuche mit Intervall [links, rechts).

Binäre Suche – rekursiv

Pseudocode (IHK-Stil):

```
Funktion binarySearchRekursiv(werte: Array<Integer>, x: Integer, links: Integer, rechts: Integer)
    if links > rechts dann
        return -1
    end if
    mitte = (links + rechts) / 2
    if werte[mitte] == x dann
        return mitte
    else if werte[mitte] < x dann
        return binarySearchRekursiv(werte, x, mitte + 1, rechts)
    else
        return binarySearchRekursiv(werte, x, links, mitte - 1)
    end if
end Funktion
```

Erklärung:

Rekursive Variante der Binärsuche: gleiche Logik, aber durch Funktionsaufrufe statt Schleife. Beachte die Abbruchbedingung (links > rechts). In Prüfungen wird meist die iterative Form bevorzugt.

Lineare Suche – Integer-Array (klassisch)

Pseudocode (IHK-Stil):

```
Funktion lineareSuche(werte: Array<Integer>, x: Integer) → Integer
    for i = 0 to länge(werte) - 1
        if werte[i] == x dann
            return i
        end if
    end for
    return -1
end Funktion
```

Erklärung:

Kein Sortieren nötig. Prüft jedes Element nacheinander.

Komplexität: $O(n)$. Vorteil: robust; Nachteil: bei großen n langsamer als Binärsuche.

Typische Anwendung: unsortierte Daten; kleine Datenmengen.

Lineare Suche – Objektliste via equals

Pseudocode (IHK-Stil):

```
Funktion lineareSucheObj(L: Liste<T>, ziel: T) → Integer
  for i = 0 to L.size() - 1
    if L.get(i).equals(ziel) dann
      return i
    end if
  end for
  return -1
end Funktion
```

Erklärung:

Objektvergleich über equals (oder passende Gleichheitslogik).

Achte darauf, ob equals sinnvoll implementiert ist (ID-basiert vs. feldweise).

Lineare Suche – Suche nach ID (Objektliste)

Pseudocode (IHK-Stil):

```
Funktion lineareSucheById(L: Liste<Kunde>, id: Integer) → Integer
    for i = 0 to L.size() - 1
        if L.get(i).getId() == id dann
            return i
        end if
    end for
    return -1
end Funktion
```

Erklärung:

Sehr praxisnah: Suche in einer Liste von Objekten nach einem eindeutigen Schlüssel (ID).
Variante: Rückgabe des Objekts statt Index. In Pseudocode reicht meist der Index.

Lineare Suche – String-Liste (case-insensitiv)

Pseudocode (IHK-Stil):

```
Funktion findeName(L: Liste<String>, s: String) → Integer
    such = lower(s)
    for i = 0 to L.size() - 1
        if lower(L.get(i)) == such dann
            return i
        end if
    end for
    return -1
end Funktion
```

Erklärung:

Bei Namen ist Groß-/Kleinschreibung häufig egal → beide Seiten in Kleinbuchstaben.
Fehlerquelle: falsche Schleifen-Grenze (letztes Element vergessen).

Lineare Suche – mit Prädikat (Function)

Pseudocode (IHK-Stil):

```
Funktion findeErstes(L: Liste<T>, test: Funktion<T → Boolean>) → Integer
  for i = 0 to L.size() - 1
    if test(L.get(i)) dann
      return i
    end if
  end for
  return -1
end Funktion
```

Erklärung:

Flexibel: test entscheidet, was als Treffer gilt (z. B. Kunde.umsatz > 1000).

Entspricht typischen Prüfungsaufgaben mit Function/Predicate.

Lineare Suche – alle Treffer sammeln

Pseudocode (IHK-Stil):

```
Funktion findeAlle(L: Liste<T>, ziel: T) → Liste<Integer>
    ergebnis = neue Liste<Integer>()
    for i = 0 to L.size() - 1
        if L.get(i).equals(ziel) dann
            ergebnis.add(i)
        end if
    end for
    return ergebnis
end Funktion
```

Erklärung:

Wenn mehrere Vorkommen existieren, gib alle Indizes zurück.

Alternative: Liste der Objekte statt Indizes. Achte auf leere Ergebnisliste statt null.